Below is a consolidated procedure based on the setup we successfully worked through today. It targets **Jetson AGX Orin running RHEL 9.4 with JetPack/L4T R36.4.0**, while compiling inside NVIDIA's l4t-jetpack:r36.4.0 container.

One naming detail is important: FFmpeg/NVmpi calls H.265 **HEVC**, so the codec names are hevc_nvmpi, not h265_nvmpi.

Build and Install FFmpeg with NVmpi on Jetson AGX Orin

Environment

Host:

```
Hardware:     NVIDIA Jetson AGX Orin
OS:           RHEL 9.4
Architecture:  aarch64
JetPack/L4T:   R36.4.0
Container:    nvcr.io/nvidia/l4t-jetpack:r36.4.0
Runtime:      Podman
```

The NVmpi implementation used here is the newer gjrtimmer/jetson-ffmpeg fork. This fork supports newer FFmpeg releases and the NvBufSurface/NvUtils API required by newer JetPack releases; support for FFmpeg 6.x–8.x is documented in the project's related upstream discussions.

Part 1 — Build Docker Image and Run Container

1.1 Create the build directory

On the RHEL host:

Bash

```
mkdir-p ~/test
cd ~/test
```

Create:

```
Dockerfile.ffmpeg
```

with:

dockerfile

```
FROM nvcr.io/nvidia/l4t-jetpack:r36.4.0

RUN apt-get update && \
   apt-get install -y --no-install-recommends \
      build-essential \
      cmake \
      git \
      pkg-config \
      yasm \
      nasm \
      autoconf \
      automake \
      libtool \
      wget \
      curl \
      ca-certificates \
      libnuma-dev && \
   rm -rf /var/lib/apt/lists/*

WORKDIR /test
```

cmake is needed for NVmpi and x265, while the compiler, assembler and autotools packages support x264 and FFmpeg builds.

1.2 Build the image

Bash

```
cd ~/test

podman build \
   -f Dockerfile.ffmpeg \
   -t l4t-jetpack:r36.4.0-ffmpeg .
```

Verify:

Bash

```
podman images
```

You should see something similar to:

```
REPOSITORY                TAG
localhost/l4t-jetpack     r36.4.0-ffmpeg
nvcr.io/nvidia/l4t-jetpack  r36.4.0
```

1.3 Run the container

Mount the host ~/test directory as /test so that source code and generated files remain after the container exits:

Bash

```
podman run --rm-it \
  --device nvidia.com/gpu=all \
  --group-add keep-groups \
  --security-optlabel=disable \
  -v${HOME}/test:/test \
  -w /test \
  localhost/l4t-jetpack:r36.4.0-ffmpeg
```

Inside the container verify:

Bash

```
gcc--version
g++--version
cmake --versiongit--version
nvcc --version
```

Because /test is bind-mounted, files generated there remain on the RHEL host even though the container uses --rm.

Part 2 — Download and Build jetson-ffmpeg / NVmpi

Do **not** use the old jocover/jetson-ffmpeg source for this R36.4 setup. The old version depends on libnvbuf_utils.so and legacy APIs such as NvBufferCreateEx, NvBufferDestroy, and NvBufferTransform. On your R36.4 system, libnvbuf_utils.so was absent while the newer NvBufSurface libraries were present. The newer fork addresses the newer Jetson buffer API.

2.1 Clone the newer NVmpi implementation

Inside the container:

Bash

```
cd /test

git clone https://github.com/gjrtimmer/jetson-ffmpeg.git
cd jetson-ffmpeg
```

2.2 Verify the JetPack multimedia libraries

On your RHEL/JetPack container, NVIDIA's libraries are under /usr/lib64/nvidia:

Bash

```
find /usr -name'libnvbufsurface.so*'2>/dev/null
find /usr -name'libnvbufsurftransform.so*'2>/dev/null
find /usr -name'libnvv4l2.so*'2>/dev/null
find /usr -name'libnvjpeg.so*'2>/dev/null
```

On your system we found:

```
/usr/lib64/nvidia/libnvbufsurface.so
/usr/lib64/nvidia/libnvbufsurftransform.so
/usr/lib64/nvidia/libnvv4l2.so
```

This RHEL-style path differs from the /usr/lib/aarch64-linux-gnu/tegra layout commonly assumed by Jetson Ubuntu builds; this exact issue has also been reported with the R36.4 container.

Verify the header:

Bash

```
ls /usr/src/jetson_multimedia_api/include/nvbufsurface.h
```

2.3 Configure NVmpi

Bash

```
cd /test/jetson-ffmpeg

mkdir-p build
cd build
```

Run:

Bash

```
cmake .. \
  -DJETSON_MULTIMEDIA_API_DIR=/usr/src/jetson_multimedia_api \
  -DJETSON_MULTIMEDIA_LIB_DIR=/usr/lib64/nvidia \
  -DCUDA_INCLUDE_DIR=/usr/local/cuda/include \
  -DCUDA_LIB_DIR=/usr/local/cuda/lib64
```

The important configure message is:

```
-- Using NvUtils API.
```

Do not continue if CMake reports any of these as NOTFOUND:

```
LIB_NVBUFSURFACE
LIB_NVBUFSURFTRANSFORM
LIB_NVJPEG
LIB_V4L2
```

2.4 Compile and install NVmpi

Bash

```
make-j$(nproc)
make install
ldconfig
```

Verify:

Bash

```
find /usr/local -name'libnvmpi*'-o-name'nvmpi.pc'
```

On our successful build:

```
/usr/local/lib/libnvmpi.so.3.10.0
/usr/local/lib/libnvmpi.so.1
/usr/local/lib/libnvmpi.so
/usr/local/lib/libnvmpi.a
/usr/local/lib/pkgconfig/nvmpi.pc
/usr/local/share/pkgconfig/nvmpi.pc
```

Verify dynamic linking:

Bash

```
ldconfig -p | grep nvmpi
```

Expected:

```
libnvmpi.so.1 => /usr/local/lib/libnvmpi.so.1
libnvmpi.so   => /usr/local/lib/libnvmpi.so
```

Configure pkg-config:

Bash

```
exportPKG_CONFIG_PATH=/usr/local/lib/pkgconfig:/usr/local/share/pkgconfig:
$PKG_CONFIG_PATH
```

Verify:

Bash

```
pkg-config --modversion nvmpi
pkg-config --cflags nvmpi
pkg-config --libs nvmpi
```

Part 3 — Download and Build libx264 and libx265

These are **software encoders**. They are not needed for NVmpi hardware acceleration itself, but including them gives FFmpeg both software and Jetson hardware encoding options and makes CPU-vs-hardware comparisons possible.

3.1 Build x264

Use HTTPS rather than the older git:// protocol, since git://git.videolan.org timed out on your network.

Bash

```
cd /test

git clone https://code.videolan.org/videolan/x264.git
cd x264
```

Configure:

Bash

```
./configure \
  --prefix=/usr/local \
  --enable-shared \
  --enable-pic
```

Compile:

Bash

```
make-j$(nproc)
```

Install:

Bash

```
make install
ldconfig
```

Verify:

Bash

```
pkg-config --modversion x264
ldconfig -p | grep x264
```

You should now have something similar to:

```
/usr/local/lib/libx264.so
```

3.2 Build x265

The x265 project has migrated its development repository to GitHub.

Clone:

Bash

```
cd /test

git clone https://github.com/Multicorewareinc/x265.git
cd x265
```

x265's documented Linux build procedure uses CMake against the source directory.

Create a separate build directory:

Bash

```
mkdir-p build/linux
cd build/linux
```

Configure:

Bash

```
cmake \
  -G"Unix Makefiles" \
  -DCMAKE_INSTALL_PREFIX=/usr/local \
  -DENABLE_SHARED=ON \
  ../../source
```

Compile:

Bash

```
make-j$(nproc)
```

Install:

Bash

```
make install
ldconfig
```

Verify:

Bash

```
pkg-config --modversion x265
ldconfig -p | grep x265
```

Update PKG_CONFIG_PATH before building FFmpeg:

Bash

```
exportPKG_CONFIG_PATH=/usr/local/lib/pkgconfig:/usr/local/share/pkgconfig:
$PKG_CONFIG_PATH
```

Verify all three dependencies:

Bash

```
pkg-config --modversion nvmpi
pkg-config --modversion x264
pkg-config --modversion x265
```

Part 4 — Download and Build FFmpeg

FFmpeg's official source is available through:

```
https://git.ffmpeg.org/ffmpeg.git
```

which is the clone URL published by the FFmpeg project.

4.1 Clone FFmpeg

For reproducibility, use FFmpeg 7.1:

Bash

```
cd /test

git clone \
  --depth=1 \
  --branch release/7.1 \
  https://git.ffmpeg.org/ffmpeg.git \
  ffmpeg
```

The selected NVmpi fork supports FFmpeg 7.1 as part of its newer FFmpeg support.

4.2 Patch FFmpeg for NVmpi

Bash

```
cd /test/jetson-ffmpeg
```

Apply the NVmpi FFmpeg patch:

Bash

```
./scripts/ffpatch.sh /test/ffmpeg
```

Then:

Bash

```
cd /test/ffmpeg
```

4.3 Configure FFmpeg

Set the package search path:

Bash

```
exportPKG_CONFIG_PATH=/usr/local/lib/pkgconfig:/usr/local/share/pkgconfig:
$PKG_CONFIG_PATH
```

Configure:

Bash

```
./configure \
  --prefix=/usr/local \
  --enable-gpl \
  --enable-libx264 \
  --enable-libx265 \
  --enable-nvmpi \
  --enable-shared \
  --enable-pic
```

The important configuration options are:

```
--enable-libx264
--enable-libx265
--enable-nvmpi
```

Compile:

Bash

```
make-j$(nproc)
```

Install:

Bash

```
make install
ldconfig
```

4.4 Verify FFmpeg configuration

Bash

```
/usr/local/bin/ffmpeg -version
```

Look for:

```
--enable-gpl
--enable-libx264
--enable-libx265
--enable-nvmpi
--enable-shared
--enable-pic
```

Verify NVmpi decoders:

Bash

```
/usr/local/bin/ffmpeg -decoders | grep nvmpi
```

Our successful build showed:

```
h264_nvmpi
hevc_nvmpi
mjpeg_nvmpi
mpeg2_nvmpi
mpeg4_nvmpi
vp8_nvmpi
vp9_nvmpi
```

Verify NVmpi encoders:

Bash

```
/usr/local/bin/ffmpeg -encoders | grep nvmpi
```

Our build showed:

```
h264_nvmpi
hevc_nvmpi
mjpeg_nvmpi
```

Also verify the software encoders:

Bash

```
/usr/local/bin/ffmpeg -encoders | grep-E'libx264|libx265'
```

Part 5 — Evaluate H.264 and H.265 NVmpi Encoding/Decoding

Run tegrastats from a **second terminal on the RHEL host**, not necessarily inside the container:

Bash

```
tegrastats --interval1000
```

GR3D_FREQ is not a direct measure of NVDEC/NVENC utilization. NVmpi uses Jetson's dedicated multimedia video engines, so low GR3D_FREQ does not imply that hardware video acceleration is inactive.

Use FFmpeg's -benchmark option to capture:

```
fps
speed
utime
stime
rtime
maxrss
```

5.1 Generate 120-second H.264 input

Create a 1920×1080, 30-fps, 120-second H.264 file at 8 Mbps:

Bash

```
cd /test/ffmpeg

/usr/local/bin/ffmpeg \
   -f lavfi \
   -i"testsrc2=size=1920x1080:rate=30" \
   -t120 \
   -c:v h264_nvmpi \
   -b:v 8M \
   input1.mp4
```

Verify:

Bash

```
/usr/local/bin/ffmpeg -i input1.mp4
```

Expected:

```
Video: h264
1920x1080
30 fps
encoder: h264_nvmpi
```

H.264 NVmpi decode test

Bash

```
/usr/local/bin/ffmpeg \
   -benchmark \
   -c:v h264_nvmpi \
   -i input1.mp4 \
   -f null -
```

Our successful test produced:

```
frame=3592
fps=462
speed=15.4x

bench:
utime=2.924s
stime=4.547s
rtime=7.781s
maxrss=71420KiB
```

The NVIDIA decoder initialization appeared as:

```
Opening in BLOCKING MODE
NvMMLiteOpen : Block : BlockType = 261
NvMMLiteBlockCreate : Block : BlockType = 261
```

Thus the 120-second 1080p30 video was decoded in approximately **7.8 seconds**, at about **462 fps / 15.4× real time**.

5.2 H.264 NVmpi encode test

Use the H.264 file as input:

Bash

```
/usr/local/bin/ffmpeg \
  -benchmark \
  -i input1.mp4 \
  -c:v h264_nvmpi \
  -b:v 8M \
  -an \
  output_h264.mp4
```

Hardware encoder initialization should include:

```
===== NvVideo: NVENC =====
NvMMLiteBlockCreate : Block : BlockType = 4
```

Our successful result was:

```
frame=3600
fps=168
speed=5.61x

bench:
utime=27.627s
stime=3.406s
rtime=21.372s
maxrss=265096KiB
```

The 120-second video was therefore encoded in approximately **21.4 seconds**, corresponding to **168 fps / 5.61× real time**.

Notice that this test used:

```
h264 (native) -> h264 (h264_nvmpi)
```

meaning:

```
H.264 input
    |
    ▼
FFmpeg software decoder
    |
    ▼
Raw YUV frames
    |
    ▼
h264_nvmpi / NVENC
```

```
   |
   ▼
output_h264.mp4
```

5.3 H.264 hardware decode + hardware encode

To test both Jetson engines:

Bash

```
/usr/local/bin/ffmpeg \
   -benchmark \
   -c:v h264_nvmpi \
   -i input1.mp4 \
   -c:v h264_nvmpi \
   -b:v 8M \
   -an \
   output_h264_hw_hw.mp4
```

The intended pipeline is:

```
H.264
  |
  ▼
h264_nvmpi decoder
  |
  ▼
raw frames
  |
  ▼
h264_nvmpi encoder
  |
  ▼
H.264
```

H.265 / HEVC Tests

Remember:

```
H.265 = HEVC
```

and FFmpeg uses:

```
hevc_nvmpi
```

not:

```
h265_nvmpi
```

5.4 Generate 120-second H.265 input

Bash

```
/usr/local/bin/ffmpeg \
  -f lavfi \
  -i"testsrc2=size=1920x1080:rate=30" \
  -t120 \
  -c:v hevc_nvmpi \
  -b:v 8M \
  input265.mp4
```

Verify:

Bash

```
/usr/local/bin/ffmpeg -i input265.mp4
```

Look for:

```
Video: hevc
1920x1080
30 fps
encoder: hevc_nvmpi
```

5.5 H.265 NVmpi decode benchmark

Bash

```
/usr/local/bin/ffmpeg \
  -benchmark \
  -c:v hevc_nvmpi \
  -i input265.mp4 \
  -f null -
```

Record:

```
frame=
fps=
speed=

bench:
utime=
stime=
rtime=
maxrss=
```

5.6 H.265 NVmpi encode benchmark

For the same test methodology used for H.264:

Bash

```
/usr/local/bin/ffmpeg \
  -benchmark \
  -i input265.mp4 \
  -c:v hevc_nvmpi \
  -b:v 8M \
  -an \
  output265.mp4
```

Confirm that you see:

```
===== NvVideo: NVENC =====
```

and:

```
encoder : ... hevc_nvmpi
```

Record the final:

```
frame=
fps=
speed=
bench: utime=... stime=... rtime=...
```

5.7 H.265 hardware decode + hardware encode

Bash

```
/usr/local/bin/ffmpeg \
  -benchmark \
  -c:v hevc_nvmpi \
  -i input265.mp4 \
  -c:v hevc_nvmpi \
  -b:v 8M \
  -an \
  output265_hw_hw.mp4
```

The complete path is:

```
input265.mp4
    |
    ▼
hevc_nvmpi decoder
    |
    ▼
raw frames
    |
    ▼
hevc_nvmpi encoder
    |
    ▼
output265_hw_hw.mp4
```

Optional Software-vs-Hardware Comparison

Because FFmpeg was built with x264/x265, you can directly compare CPU software codecs against NVmpi.

H.264 software decode

Bash

```
/usr/local/bin/ffmpeg \
  -benchmark \
  -c:v h264 \
  -i input1.mp4 \
  -f null -
```

H.264 hardware decode

Bash

```
/usr/local/bin/ffmpeg \
  -benchmark \
  -c:v h264_nvmpi \
  -i input1.mp4 \
  -f null -
```

H.264 software encode

Bash

```
/usr/local/bin/ffmpeg \
  -benchmark \
  -i input1.mp4 \
  -c:v libx264 \
  -b:v 8M \
  -an \
  software_h264.mp4
```

H.264 hardware encode

Bash

```
/usr/local/bin/ffmpeg \
  -benchmark \
  -i input1.mp4 \
  -c:v h264_nvmpi \
  -b:v 8M \
  -an \
  hardware_h264.mp4
```

Likewise for H.265:

Bash

```
# Software
/usr/local/bin/ffmpeg \
  -benchmark \
  -i input265.mp4 \
  -c:v libx265 \
  -b:v 8M \
  -an \
  software_h265.mp4
```

versus:

Bash

```
# Hardware
/usr/local/bin/ffmpeg \
   -benchmark \
   -i input265.mp4 \
   -c:v hevc_nvmpi \
   -b:v 8M \
   -an \
   hardware_h265.mp4
```

For each test, compare **fps**, **speed**, **rtime**, CPU utilization from htop/tegrastats, and memory usage from maxrss. x265 is the open-source HEVC software encoder, while FFmpeg itself provides the framework into which these codec libraries and NVmpi are integrated.

This gives you a complete reproducible build path:

```
RHEL 9.4 / Jetson AGX Orin
        │
        ▼
l4t-jetpack:r36.4.0
        │
        ├── x264
        ├── x265
        ├── libnvmpi
        │     └── NvBufSurface / NvUtils
        │
        ▼
     FFmpeg 7.1
        │
  ┌─────┴──────┐
  │            │
Software      Hardware
libx264       h264_nvmpi
libx265       hevc_nvmpi
```

The FFmpeg project publishes its Git source directly, x265's current development repository is on GitHub, and the NVmpi fork used here is specifically the newer branch of the Jetson-FFmpeg work needed for modern FFmpeg/Orin environments.